Case Study 15 — Vibronic Tabular Branch

Pre-computed bandshapes through the experimental input branch

Contents

1 Case Study 15 — Vibronic Tabular Branch

Demonstrate that the tool is not locked to Gaussian broadening. A vibronic (Franck–Condon) or any other pre-computed absorption spectrum can be fed through the experimental/spreadsheet (CSV/XLSX) input branch and runs through the identical absorbed-flux, absorbed-fraction, and shape-overlap descriptor pipeline.

The key insight: `overlap-calculator` treats every experimental tabular input as absorbance $A(\lambda)$ directly and does not broaden it further. Any bandshape — vibronic progression, empirical spectrum, or computationally generated Franck–Condon envelope — is handled correctly, with no additional flags required.

The user-selectable knob is: use `input_type = "experimental"` in the manifest (or supply a CSV/XLSX to `generate-input`).

1.1 What you will produce

Folder	By	Contents
`output/cli/tables/`	CLI	Result tables (CSV / JSON / XLSX)
`output/cli/plots/`	CLI	TIFF plots per sample, light source, and (sample, light) pair
`output/api/`	API	Same `tables/` and `plots/` tree, extracted from the ZIP
`output/api/analysis_outputs.zip`	API	Raw ZIP response

Light sources used: **AM1.5G + LED-B4**.

1.2 1. Prerequisites

If you have never run any case study before, follow Case Study 01 §1 Prerequisites once to install the package, the `requests` library, and either the Python or Docker option for the HTTP API.

1.3 2. Inputs

This case study uses the same bundled experimental workbook as Case Studies 02, 03, and 11. The `input.json` selects 2 absorbance series from the PhotochemCAD-derived spreadsheet. Each series is a real UV/Vis absorption spectrum measured in solution — it captures the true bandshape including any vibronic structure already resolved by the spectrometer.

Citation. The bundled workbook is derived from M. Taniguchi & J. S. Lindsey, *Photochem. Photobiol.* **94** (2018) 290–327. See the project README for the full citation.

1.4 3. Run with the CLI

```
overlap-calculator analyze --input case_studies/15_vibronic_tabular_branch/input.json --out
↪ case_studies/15_vibronic_tabular_branch/output/cli --default-light-sources AM15G,LEDB4 --plot-dpi 400
```

After the run the `source_type` column in `results.*` reads `experimental` for every row, confirming that no broadening was applied.

1.5 4. Run with the HTTP API

```
curl -X POST http://localhost:8000/analyze -F
↪ "files=@input/files/organic_uvvis_photochemcad_dataset.xlsx" -F "default_light_sources=AM15G,LEDB4"
↪ -F "plot_dpi=400" --output case_studies/15_vibronic_tabular_branch/output/api/analysis_outputs.zip
```

1.6 5. Drive both interfaces from one Python script

```
python case_studies/15_vibronic_tabular_branch/submit.py
```

1.7 6. Expected output layout

```
case_studies/15_vibronic_tabular_branch/output/
+-- cli/
|   +-- run_manifest.json
|   +-- tables/results.{csv,json,xlsx}        (2 series x 2 light sources = 4 rows)
|   +-- tables/results_timings.{csv,json,xlsx}
|   +-- tables/descriptor_summary.{csv,xlsx}
|   +-- tables/ranking_by_light_source__<metric>.{csv,json,xlsx}
|   +-- tables/ranking_by_sample__<metric>.{csv,json,xlsx}
|   +-- plots/
+-- api/
    +-- analysis_outputs.zip
    +-- tables/...
    +-- plots/...
```

1.8 7. What this demonstrates

- The absorbed-flux, absorbed-fraction, and shape-overlap descriptor pipeline is identical for theoretical and experimental inputs. The only difference is that experimental inputs skip the broadening step.
- A vibronic spectrum stored in a CSV or XLSX file (wavelength column + absorbance column) is all

that is needed. No extra flags are required.
- The `shape_overlap` metric in particular is broadening-agnostic: it compares the max-normalised shape of the supplied spectrum against the max-normalised light-source spectrum regardless of how the absorption bandshape was generated.

1.9 8. Related case studies

- Case Study 02 — Experimental Only
- Case Study 03 — Mixed Theory + Experiment
- Case Study 13 — Marcus–Hush Width
- Case Study 14 — Calibration Block